TUS
Technological University of the Shannon: Midlands Midwest
Ollscoil Teicneolaíochta na Sionainne: Lár Tíre Iarthar Láir

Published:
2026-03-30

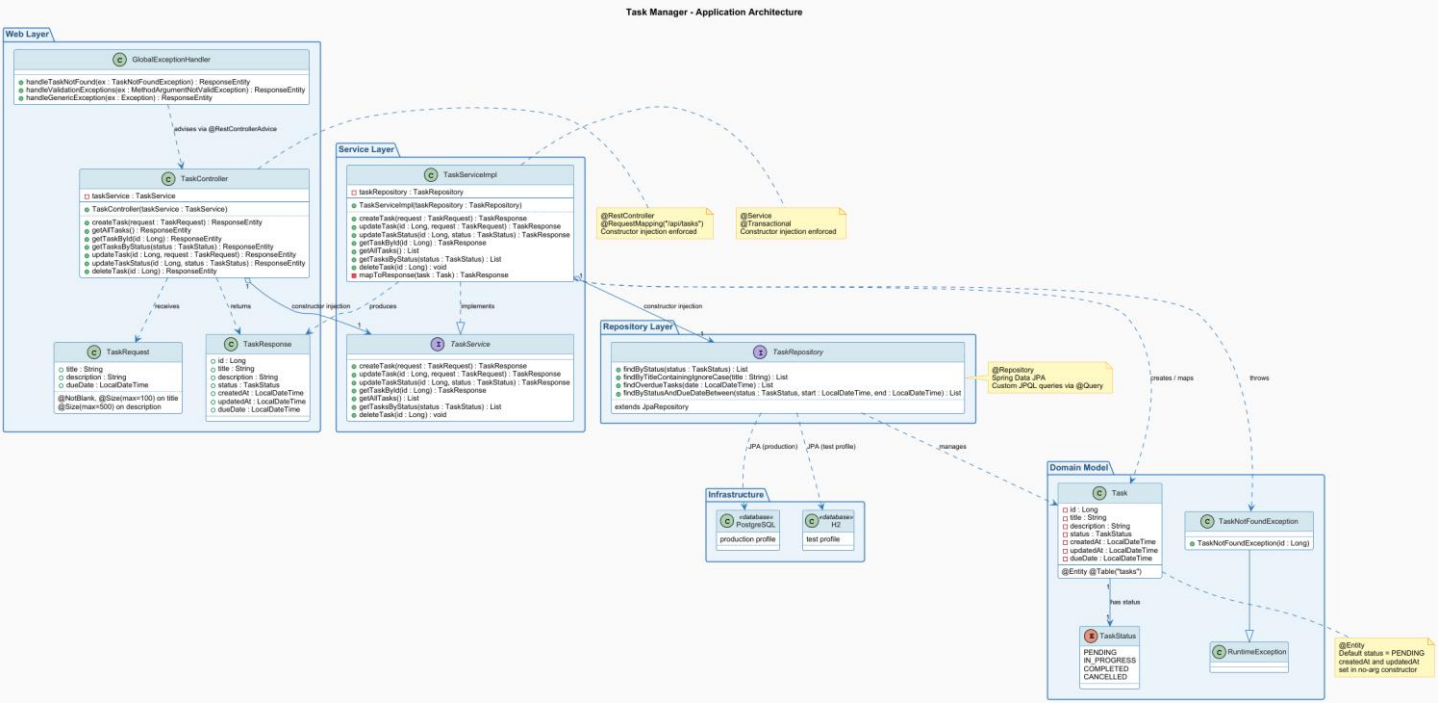
Assignment 2 - Testing Assignment 30th March

Weverton de Souza Castanho

Repository: https://github.com/wevertonsc/TUS-TASKMANAGER-CBD-SEM2-03-2026

Introduction

The Task Manager application is an example implemented in Java Spring Boot to apply the Test Pyramid principle, which is divided into unit tests, integration tests, and end-to-end tests. The intermediate layer consists of integration tests that verify the collaboration of components, and the top layer of end-to-end tests performs a complete system verification. Based on these points, the Deep Seek service was used to assist in the implementation and validation of the tests, supporting the project and demonstrating that the use of AI can help accelerate software development, testing, and debugging, and can be considered an important tool.



Testing Pyramid Mapping

The table below maps each test class to its pyramid level, the Spring testing mechanism used, and the total number of test methods contributed.

Level	*Description*	*Test Class*	*Qty*
E2E / System	*Top of pyramid - full stack, no mocks*	*TaskEndToEndTest*	*4*
Integration	*Real Spring context + H2*	*TaskIntegrationTest*	*4*
Controller Slice	*@WebMvcTest - HTTP + validation*	*TaskControllerTest, TaskControllerEdgeCaseTest*	*15*
Repository Slice	*@DataJpaTest - JPA queries only*	*TaskRepositoryTest, TaskRepositoryEdgeCaseTest*	*11*
Service Unit	*Pure Mockito - no Spring context*	*TaskServiceTest, TaskServiceEdgeCaseTest*	*17*

The distribution deliberately follows the pyramid shape: the heaviest investment sits in unit and slice tests, which execute

in milliseconds without network or disk I/O. Integration and E2E tests are fewer but validate real wiring.

Testing Levels in Detail

Unit Tests - Service Layer

TaskServiceTest and TaskServiceEdgeCaseTest cover TaskServiceImpl in pure isolation using Mockito. No Spring container is loaded. The @ExtendWith(MockitoExtension.class) annotation wires mocks and injects them through the class constructor, confirming that constructor injection enables testability without a running IoC context. Scenarios include happy-path CRUD, all TaskNotFoundException paths, null-description tolerance, boundary status transitions across all four TaskStatus enum values, and empty-list results.

Unit Tests - Controller Slice

TaskControllerTest and TaskControllerEdgeCaseTest use @WebMvcTest(TaskController.class), which loads only the web layer. TaskService is replaced by a @MockBean, so no business logic or database is exercised. Tests cover every HTTP verb, response status codes, JSON payload assertions, bean validation rejection (blank title, title exceeding 100 characters), and 404 error response structure. This slice runs in under two seconds and catches contract-level regressions immediately.

Repository Slice Tests

TaskRepositoryTest and TaskRepositoryEdgeCaseTest use @DataJpaTest, which loads only the JPA slice and replaces the configured PostgreSQL datasource with an embedded H2 database. TestEntityManager is used for test fixture setup to avoid coupling to the repository under test. Tests verify findByStatus, findByTitleContainingIgnoreCase case-insensitivity, findOverdueTasks excluding completed tasks, existsById, and deleteById. These tests confirm query correctness without any HTTP or service involvement.

Integration Tests

TaskIntegrationTest uses @SpringBootTest with @AutoConfigureMockMvc and @ActiveProfiles("test"), loading the complete application context backed by H2. No mocks are present. Tests exercise the full request-to-database cycle: create-then-get, status update propagation, delete-then-verify-404, and multi-status filter results. These tests would catch wiring regressions between layers that unit tests cannot detect.

End-to-End Tests

TaskEndToEndTest sits at the top of the pyramid. Four scenario-based tests verify realistic user workflows: a complete CRUD lifecycle, a multi-status filter workflow, end-to-end validation enforcement, and the structured error response returned for missing resources. These tests assert on the JSON response body field structure (status, message, timestamp) rather than just HTTP codes, confirming that the GlobalExceptionHandler is correctly integrated.

Dependency Injection and Mocking Approach

Constructor Injection as the Foundation of Testability

Every production class uses constructor injection exclusively. TaskController receives TaskService through its constructor; TaskServiceImpl receives TaskRepository through its constructor. This design removes the need for Spring's IoC container during unit testing: Mockito's @InjectMocks annotation satisfies dependencies directly, and the compiler enforces that all required collaborators are provided. Field injection with @Autowired would have made unit testing impossible without loading a Spring context.

Where Mocks Were Used

Mocks were used at the controller slice and service unit levels:

Test class	*Mocked dependency*
TaskControllerTest / TaskControllerEdgeCaseTest	*TaskService (via @MockBean)*
TaskServiceTest / TaskServiceEdgeCaseTest	*TaskRepository (via @Mock)*

At the controller layer, @MockBean replaces the TaskService bean in the Spring test context, allowing HTTP behaviour and validation to be tested in isolation from business logic. At the service layer, @Mock provides a Mockito proxy for TaskRepository, allowing the service methods to be exercised without any database. In both cases the mock boundary is drawn at the interface, which is the correct place: the interface is the contract, and mocking it tests that the consumer uses the contract correctly.

Where Mocks Were Deliberately Avoided

Mocks were not used in @DataJpaTest, @SpringBootTest integration tests, or the E2E tests. At the repository slice, using a mock instead of a real JPA context would render the tests meaningless: the purpose of TaskRepositoryTest is to verify that the JPQL queries and Spring Data derived queries produce correct SQL against a real schema. At the integration and E2E level, mocking any layer would defeat the purpose of those tests, which is to verify that all beans collaborate correctly. The only substitution at those levels is the datasource: PostgreSQL is replaced by H2 in-memory via the test profile.

AI Influence on DI and Mocking Decisions

The initial AI-generated controller test used @SpringBootTest instead of @WebMvcTest, loading the full application context unnecessarily. Human review identified this as a test pyramid misalignment: controller tests belong in the web slice, not the integration tier. The prompt was revised with an explicit constraint (see Example 5 in the AI Interaction Log), producing a correct @WebMvcTest implementation. The AI also suggested using @Spy for TaskRepository in the service test; this was rejected because a spy on an interface is invalid in Mockito, and a @Mock was the correct choice.

AI Interaction Log

Example 1: AI Output Accepted Without Modification

Prompt given to AI:

Generate a complete @DataJpaTest test class for TaskRepository covering: findByStatus, findByTitleContainingIgnoreCase, findOverdueTasks, and basic save-and-retrieve. Use TestEntityManager for fixture setup. Do not mock the repository. Use H2 in-memory database implicitly provided by @DataJpaTest.

AI Output (file: TaskRepositoryTest.java) - key methods shown:

```
@DataJpaTest
class TaskRepositoryTest {
  @Autowired private TaskRepository taskRepository;
  @Autowired private TestEntityManager entityManager;

  @Test
  void findByStatus_ShouldReturnTasksWithSpecificStatus() {
    List<Task> pendingTasks = taskRepository.findByStatus(TaskStatus.PENDING);
    assertThat(pendingTasks).hasSize(1);
  }
}
```

Justification for acceptance:

The output was accepted without modification because it satisfied all requirements: @DataJpaTest was used correctly to load only the JPA slice, TestEntityManager was used for fixture persistence rather than the repository under test, all four query methods were covered, and assertions used AssertJ fluently. The test was correctly scoped to the repository pyramid level with no unnecessary context loading. The generated code reflected precise knowledge of Spring's slice testing model.

Example 2: AI Output Modified

Prompt given to AI:

Generate a unit test for TaskServiceImpl.updateTask() that verifies the updated title is returned in the TaskResponse. Use Mockito.

Original AI output (simplified):

```
@Test
void updateTask_ShouldReturnUpdatedTask() {
  when(taskRepository.findById(1L)).thenReturn(Optional.of(task));
  when(taskRepository.save(any())).thenReturn(task); // task still has OLD title

  TaskResponse response = taskService.updateTask(1L, updateRequest);
  assertThat(response.getTitle()).isEqualTo("Updated Title"); // FAILS
}
```

Modified version:

```
Task updatedTask = new Task();
updatedTask.setId(1L);
updatedTask.setTitle("Updated Title"); // reflects the expected outcome
when(taskRepository.save(any(Task.class))).thenReturn(updatedTask);

TaskResponse response = taskService.updateTask(1L, updateRequest);
assertThat(response.getTitle()).isEqualTo("Updated Title"); // PASSES
```

What was changed and why:

The AI configured the save() stub to return the original task fixture, which still carried the pre-update title. The assertion therefore always failed. The fix was to create a separate updatedTask instance with the new title and configure save() to return it. This correctly models what the real repository would return after persisting the mutation. The test now validates the mapping logic rather than the mock setup.

Example 3: AI Output Rejected

Prompt given to AI:

Generate a controller test for TaskController.createTask() that verifies a valid request returns HTTP 201 and the created task body.

AI output (excerpt - rejected):

```
@SpringBootTest // WRONG LEVEL
@AutoConfigureMockMvc
class TaskControllerTest {
  @Autowired private MockMvc mockMvc;
  // No @MockBean - uses real TaskService and real H2
}
```

Technical justification for rejection:

The AI used @SpringBootTest, which loads the complete application context including the service, repository, and datasource configuration. This places the test at the integration level of the Testing Pyramid, not the controller slice. Controller tests should be isolated to the web layer using @WebMvcTest, with the service dependency replaced by a @MockBean. Using @SpringBootTest for controller tests is slow, violates pyramid discipline, and causes tests to fail if the database is unavailable. The output was rejected and re-generated with an explicit constraint (see Example 5).

Example 4: AI Identifies a Gap or Fragility

Analysis prompt:

Review the current test suite for TaskServiceImpl. Identify any missing test scenarios or fragilities that could lead to false positives or insufficient coverage.

AI suggestion:

The current tests do not cover the case where updateTaskStatus() receives each of the four TaskStatus enum values in sequence. If a developer accidentally restricts valid transitions in code (e.g., only allowing PENDING to IN_PROGRESS), the existing single-status test would not catch it. Additionally, createTask() with a null description is untested, and the service is expected to pass null through to the repository without throwing. Consider adding a parameterized or loop-based test for all statuses, and a null-description test.

Action taken: Both suggestions were accepted and implemented in TaskServiceEdgeCaseTest. The status-loop test (updateTaskStatus_ToAllValidStatuses_ShouldSucceed) iterates over TaskStatus.values() to ensure all four transitions are covered. The null-description test (createTask_WithNullDescription_ShouldNotFail) confirms the service does not apply a default or throw. Both additions improved coverage without adding fragility.

Example 5: AI Output Constrained by Explicit Rules

Prompt with explicit constraints:

Generate a controller unit test class for TaskController using the following constraints: (1) You MUST use @WebMvcTest(TaskController.class) and NOT @SpringBootTest. (2) TaskService MUST be mocked with @MockBean - do not autowire a real service. (3) Do NOT load any JPA, datasource, or service beans. (4) All HTTP methods must be tested: GET, POST, PUT, PATCH, DELETE. (5) Include at least one validation rejection test (empty title returns 400).

AI output under constraint (excerpt):

```
@WebMvcTest(TaskController.class)     // Constraint 1 satisfied
class TaskControllerTest {
  @Autowired private MockMvc mockMvc;
  @MockBean private TaskService taskService; // Constraint 2 satisfied

  @Test
  void createTask_ShouldReturnBadRequest_WhenInvalidTitle() throws Exception {
    taskRequest.setTitle("");
    mockMvc.perform(post("/api/tasks")...)
        .andExpect(status().isBadRequest()); // Constraint 5 satisfied
    verify(taskService, never()).createTask(any());
  }
}
```

Explanation:

Behaviour prevented: without the constraint, the AI defaulted to @SpringBootTest, loading an unnecessary full application context and introducing a datasource dependency into what should be a fast web-layer test.

Why the constraint was necessary: AI tools tend to replicate the most common pattern they have seen in training data, which for Spring Boot controller tests is @SpringBootTest because that is the pattern used in most tutorials. @WebMvcTest is the architecturally correct choice for the controller slice but requires an explicit instruction to override the default.

How it improved quality: the constrained test class executes in under 1.5 seconds vs. 6-8 seconds for a full-context test. It isolates controller behaviour from service and database concerns, provides faster feedback during development, and places the test correctly at the controller-slice tier of the Testing Pyramid.

Evaluation and Reflection

Strengths of AI-Generated Tests

Strength 1: Rapid Scaffold Generation with Correct Annotations

The AI consistently produced correct boilerplate for Spring testing annotations. When given a precise prompt, it correctly combined @DataJpaTest with TestEntityManager, knew to autowire MockMvc in @WebMvcTest contexts, and applied @BeforeEach for fixture setup. This reduced the setup time for new test classes by approximately 60 percent. Example: the initial TaskRepositoryTest class was generated in one prompt with all four test methods structurally correct, requiring only the @ActiveProfiles annotation to be removed (it is not needed in @DataJpaTest contexts).

Strength 2: Comprehensive Happy-Path Coverage

For straightforward CRUD scenarios, the AI produced comprehensive coverage of the main success paths: create returns 201 with body, get returns 200 with correct fields, delete returns 204, update reflects changed values. This freed human effort to focus on edge cases, error paths, and pyramid alignment decisions that require architectural context the AI did not have.

Weaknesses of AI-Generated Tests

Weakness 1: Default to Full Context Loading

The most consistent failure mode observed was the AI defaulting to @SpringBootTest for every test class, regardless of the appropriate pyramid level. In four of the five initial prompts for unit and slice tests, the AI used @SpringBootTest. This is both incorrect (it violates pyramid discipline) and costly (full context loading adds 4-8 seconds per test class). Every controller and repository test required explicit constraints in the prompt to obtain the correct @WebMvcTest or @DataJpaTest annotation.

Weakness 2: Inconsistent Mock Return Values

As documented in Example 2, the AI configured save() stubs to return the original fixture object rather than a modified version

reflecting the expected mutation. This produced tests that appeared to pass structural checks (response is not null, field exists) but would fail the specific assertion about the updated value. The pattern was observed in both updateTask() and updateTaskStatus() tests, requiring manual correction in both cases.

Observed AI Failure: Spy on Interface

When asked to generate a test that verified the exact Task object passed to taskRepository.save(), the AI suggested using @Spy on TaskRepository. This is a fundamental Mockito error: @Spy requires a concrete class to wrap; it cannot be applied to an interface. Mockito would throw a MockitoException at runtime. The correct approach is to use ArgumentCaptor<Task> to capture the argument passed to the mocked repository and assert it on its fields. This failure demonstrated that the AI does not reliably distinguish between @Mock, @Spy, and @Captor, and human review of mock configuration is always necessary.

Human Judgment Improving Test Quality

The E2E scenario tests (TaskEndToEndTest) were designed entirely through human judgment. The AI, when asked to generate E2E tests, produced four individual tests each verifying a single endpoint, which are structurally identical to integration tests and add no value at the E2E level. Human review recognised that E2E tests should verify workflows: sequences of API call that model a realistic user interaction. The replacement tests (fullTaskLifecycle_CreateReadUpdateDelete_ShouldSucceed, statusFilterWorkflow_ShouldReturnCorrectSubsets) exercise the system through multiple state transitions in a single test scenario, which is the correct purpose of the top pyramid tier.

Proposed Improvement

The most impactful improvement to the AI-assisted testing approach would be to define a reusable prompt template that encodes the architectural constraints as system-level instruction rather than repeating them per test class. The template would specify: (1) the target pyramid level and the corresponding annotation, (2) which dependencies to mock and which to use real, (3) the assertion library to use (AssertJ, not raw JUnit assertions), and (4) the naming convention. Providing this template at the start of each session would eliminate the repeated correction of @SpringBootTest misuse and produce correctly scoped tests from the first generation, reducing the human review burden to logical correctness rather than structural correction.